MediCheck

AI-Powered Healthcare Bill Accuracy & Dispute Assistant

Requirements & Design Document

Quantic MSSE Capstone | Sprint 1

Shifali Srivastava & Nadia van der Merwe

Version 1.0 | March 2026

Table of Contents

1. **Personas**
   - Persona 1 — Sarah: First-Time Patient User
   - Persona 2 — Marcus: Repeat Patient / Disputer
   - Persona 3 — Jordan: Clinic Integration Developer
   - Persona Summary
2. **User Stories**
   - Persona 1 — Sarah (US-001 to US-006)
   - Persona 2 — Marcus (US-007 to US-012)
   - Persona 3 — Jordan (US-013 to US-018)
3. **User Journey**
   - Phase 1: Discovery & Onboarding
   - Phase 2: Upload & Extraction
   - Phase 3: Analysis & Results
   - Phase 4: Action & Resolution
   - Emotional Journey
4. **System Requirements**
   - 4.1 Functional Requirements (FR-01 to FR-27)
   - 4.2 Non-Functional Requirements (NFR-01 to NFR-27)
5. **Requirements Traceability Matrix**
   - 5.1 Functional Requirements → User Stories
   - 5.2 Non-Functional Requirements → User Stories
   - 5.3 Reverse Matrix — User Story → Requirements
   - 5.4 Gap Analysis

1. Personas

Three personas drive the MediCheck product design. Each maps to a distinct demo scenario and a distinct set of functional requirements.

Persona 1 — Sarah: First-Time Patient User

Field	Detail
Age	34
Occupation	Office Manager
Income	$65,000 / year
Location	Charleston, South Carolina
Medical Billing Knowledge	None — never disputed a bill before
Technology Comfort	High — uses smartphone apps daily, comfortable uploading documents

Pain Points:

- Received a $2,400 bill from MUSC Health after routine knee arthroscopy
- Suspects errors but doesn't understand CPT codes or medical terminology
- Spent 2 hours Googling medical billing, still confused
- Doesn't know if $2,400 is reasonable or excessive
- Afraid to call hospital billing — intimidating, long hold times

Goals:

- Find errors quickly without learning medical billing
- Understand which charges are questionable in plain English
- Get evidence to dispute errors — letter template, regulatory citations
- Avoid overpaying for medical care

> "I know something's wrong with this bill, but I have no idea where to start. I just want someone to tell me if I'm being ripped off."

Why Sarah matters: She represents the majority of MediCheck's target users — patients with no billing expertise who receive a bill that feels wrong. Maps to Demo Scenario B: MUSC Health / Ambetter HMO, facility fee at 340% Medicare rate and EOB mismatch.

Persona 2 — Marcus: Repeat Patient / Disputer

Field	Detail
Age	52
Occupation	Small Business Owner (HVAC company)

Field	Detail
Income	$95,000 / year
Location	Greenville, South Carolina
Medical Billing Knowledge	Moderate — fought errors on previous bills, won $600 dispute
Technology Comfort	Medium-High — uses QuickBooks, comfortable with spreadsheets and reports

Pain Points:

- Spent 8 hours manually checking his last Prisma Health bill line-by-line against Medicare rates
- Found $600 in errors (duplicate CT scan, upcoded office visit) through manual work alone
- Knows billing errors are common but manual verification is exhausting and unsustainable
- Wants to check bills proactively before paying, not after the fact
- Needs legal citations and regulatory references to give his disputes credibility

Goals:

- Automate error detection to save time on every new bill
- Compare charges to Medicare rates instantly without spreadsheet work
- Get regulatory citations (No Surprises Act, CMS guidelines) for legal leverage
- Download a ready-to-send dispute letter with specific errors and dollar amounts cited

> "I know how to fight these errors, but I don't have 8 hours to spend on every bill. I need a tool that does the heavy lifting for me."

Why Marcus matters: Represents power users who know the system can be wrong and need automation to make disputing practical. Maps to Demo Scenario C: Prisma Health / Molina Healthcare, upcoded diabetes follow-up visit triggering Module 2.

Persona 3 — Jordan: Clinic Integration Developer

Field	Detail
Role	Full-Stack Developer
Employer	Atrium Health patient portal team, Charlotte, NC
Team Size	4 developers
Technical Knowledge	High — REST APIs, JSON, React, Node.js, PostgreSQL
Technology Stack	Node.js backend, React frontend, PostgreSQL database

Pain Points:

- Patients call the Atrium Health billing team to dispute charges — staff are overwhelmed
- No existing tool provides structured, machine-readable billing error output
- Building error detection in-house would require medical billing domain expertise the team doesn't have
- Needs clean REST endpoints, consistent JSON response schemas, and predictable error codes

- Must handle edge cases gracefully — timeouts and malformed input cannot crash the portal

Goals:

- Call POST /upload, POST /confirm, and POST /analyse from the clinic's React portal
- Receive structured JSON with error type, severity, affected line items, and dollar impact
- Display RAG-grounded plain-English explanations and dispute letter download in patient portal
- Handle Service 2 and Service 3 timeouts gracefully — never expose a raw 500 error
- Complete integration within a single sprint using clear docs and example payloads

> "If the endpoints are consistent and the JSON is predictable, I can wire this into our portal in a sprint. I just need docs and example responses — I'll handle the rest."

Why Jordan matters: Represents the developer API perspective. Maps to Demo Scenario A: Atrium Health / BCBS SC PPO, all four modules firing — NSA violation, duplicate surgical tray charge, upcoded anaesthesia.

Persona Summary

	Sarah	Marcus	Jordan
Type	First-time patient	Repeat patient / disputer	Clinic integration developer
Location	Charleston, SC	Greenville, SC	Charlotte, NC
Drives	Plain-English output, upload UX, OCR confirmation	Medicare rate comparison, regulatory citations, dispute letter	JSON schema, timeout handling, /health endpoint, docs
Demo Scenario	Scenario B (MUSC Health, Ambetter)	Scenario C (Prisma Health, Molina)	Scenario A (Atrium Health, BCBS SC)
Priority	Must Have	Must Have	Should Have

2. User Stories

18 user stories across 3 personas. Format: As a / I want / so that. Each story includes 4–6 acceptance criteria, MoSCoW priority, story points (Fibonacci), sprint assignment, and epic linkage.

ID	Persona	Title	Priority	Points	Sprint	Epic
US-001	Sarah	Upload a medical bill	Must Have	5	2	OCR & Field Confirmation
US-002	Sarah	Review extracted fields	Must Have	5	2	OCR & Field Confirmation
US-003	Sarah	Results within 30 seconds	Must Have	3	3	Error Detection
US-004	Sarah	Errors in plain English	Must Have	5	4	RAG & Dispute Letters
US-005	Sarah	Download dispute letter	Must Have	5	4	RAG & Dispute Letters
US-006	Sarah	Data privacy	Must Have	2	2	Three-Service Infra
US-007	Marcus	Medicare rate comparison	Must Have	5	3	Error Detection
US-008	Marcus	Detect duplicate charges	Must Have	3	3	Error Detection
US-009	Marcus	Regulatory citations	Must Have	3	4	RAG & Dispute Letters
US-010	Marcus	EOB reconciliation	Should Have	8	3	Error Detection
US-011	Marcus	NSA violations	Should Have	5	3	Error Detection
US-012	Marcus	Download results PDF	Should Have	3	4	RAG & Dispute Letters
US-013	Jordan	External app endpoints	Should Have	5	2	Three-Service Infra
US-014	Jordan	Structured JSON response	Should Have	3	3	Error Detection
US-015	Jordan	Service 3 timeout handling	Must Have	3	3	Three-Service Infra
US-016	Jordan	Health endpoint monitoring	Must Have	2	2	Three-Service Infra
US-017	Jordan	API documentation	Should Have	3	5	Final Submission

ID	Persona	Title	Priority	Points	Sprint	Epic
US-018	Jordan	CI/CD pipeline stability	Should Have	3	2	Three-Service Infra
		Total	Must Have: 10 | Should Have: 8	71		

Persona 1 — Sarah: First-Time Patient

Sarah is a 34-year-old office manager in Charleston, SC. No medical billing knowledge. Received a $2,400 bill from MUSC Health after routine knee arthroscopy.

US-001 — Upload a medical bill for analysis

As a first-time patient user, I want to upload my medical bill as a PDF or photo, so that the system can extract its charges and analyse them for errors without me needing to type anything manually.

Priority: Must Have | **Points:** 5 | **Sprint:** 2 | **Epic:** OCR & Field Confirmation

Requirements: FR-01, FR-02

Acceptance Criteria:

- ☐ Drag-and-drop upload zone accepts PDF, JPG, and PNG files
- ☐ File size validation rejects files over 10 MB with a clear error message
- ☐ Upload progress bar is visible during file transfer
- ☐ On successful upload, user sees a confirmation message and transitions to field confirmation screen
- ☐ On failure (network timeout, corrupt file), user sees an actionable error message with a retry option

US-002 — Review and correct extracted bill fields

As a first-time patient user, I want to see the data extracted from my bill in an editable table with confidence scores, so that I can fix any OCR errors before analysis runs and trust that the findings are based on accurate input.

Priority: Must Have | **Points:** 5 | **Sprint:** 2 | **Epic:** OCR & Field Confirmation

Requirements: FR-06, FR-07, FR-08

Acceptance Criteria:

- ☐ Extracted fields displayed in table: Field Name, Extracted Value, Confidence Score, Corrected Value
- ☐ Fields with confidence below 80% are highlighted in yellow
- ☐ Every field is editable inline — clicking a value opens an edit input
- ☐ Date fields validate to MM/DD/YYYY format; amount fields validate to $XX.XX format
- ☐ "Confirm & Analyse" button is disabled until the user has reviewed all highlighted fields
- ☐ Confirmed data is stored in PostgreSQL before analysis begins

US-003 — Receive analysis results within 30 seconds

As a first-time patient user, I want to see my bill analysis results within 30 seconds of clicking "Analyse", so that I don't abandon the tool while waiting and lose confidence in the process.

Priority: Must Have | **Points:** 3 | **Sprint:** 3 | **Epic:** Knowledge Base & Error Detection

Requirements: NFR-01, NFR-02

Acceptance Criteria:

- ☐ Analysis results returned to the UI within 30 seconds for bills up to 20 pages
- ☐ A progress indicator shows which detection module is currently running
- ☐ If Service 3 (RAG) times out, error report still loads with core data and a "Retry explanations" option
- ☐ If no errors detected, a positive confirmation message is shown with a summary of what was checked

US-004 — Understand billing errors in plain English

As a first-time patient user, I want to see each detected error explained in plain English with the specific line items affected, so that I can understand what went wrong without needing to know medical billing terminology.

Priority: Must Have | **Points:** 5 | **Sprint:** 4 | **Epic:** RAG Integration & Dispute Letters

Requirements: FR-16, FR-18

Acceptance Criteria:

- ☐ Each error card displays: error type, affected line item numbers, estimated dollar impact, plain-English explanation
- ☐ Explanations contain no unexplained medical jargon — CPT codes accompanied by plain descriptions
- ☐ Each explanation includes at least one regulatory or source citation (CMS, Congress.gov)
- ☐ Error cards colour-coded: red for high severity (>$100), amber for medium ($50–$100), green for informational (<$50)
- ☐ A savings summary card at the top shows total errors found and total potential savings

US-005 — Download a dispute letter with regulatory citations

As a first-time patient user, I want to download a professionally formatted dispute letter citing the specific errors found and relevant regulations, so that I can submit it to the hospital or insurer without needing to write it myself or hire a lawyer.

Priority: Must Have | **Points:** 5 | **Sprint:** 4 | **Epic:** RAG Integration & Dispute Letters

Requirements: FR-21, FR-22, FR-23

Acceptance Criteria:

- ☐ "Generate Dispute Letter" button appears on the error report page when at least one error is detected
- ☐ Letter includes: patient name, provider name, bill reference number, date of service, each error with line items, dollar amounts, and regulatory citations
- ☐ Letter is available for download in both Word (.docx) and PDF formats
- ☐ Word version is fully editable so the user can add personal details before sending
- ☐ Letter generation completes within 10 seconds of clicking the button

US-006 — Trust that my data is private

As a first-time patient user, I want to be clearly informed that my medical bill is not stored permanently, so that I feel safe uploading sensitive personal health information without creating an account.

Priority: Must Have | **Points:** 2 | **Sprint:** 2 | **Epic:** Three-Service Infrastructure

Requirements: FR-24, FR-27, NFR-06, NFR-08

Acceptance Criteria:

- ☐ Homepage and upload page display a clear privacy statement: "Your bill is processed securely and deleted after 24 hours"
- ☐ No account or email address is required to use the tool
- ☐ All communication between client and server uses HTTPS
- ☐ No PHI is retained in PostgreSQL after the session expires (24-hour auto-delete)

Persona 2 — Marcus: Repeat Patient / Disputer

Marcus is a 52-year-old small business owner in Greenville, SC. Has won billing disputes before and needs automation to make checking every bill practical.

US-007 — Compare billed charges to Medicare rates instantly

As a repeat patient user who disputes billing errors, I want to see each charge compared to the Medicare rate for the same procedure, so that I can immediately identify which items are outliers without spending hours on a Medicare fee schedule spreadsheet.

Priority: Must Have | **Points:** 5 | **Sprint:** 3 | **Epic:** Knowledge Base & Error Detection

Requirements: FR-12, FR-16, FR-17

Acceptance Criteria:

- ☐ Error report shows the Medicare rate alongside the billed amount for every line item where comparison is possible
- ☐ Items exceeding 300% of the Medicare rate are automatically flagged as outliers
- ☐ The ratio is displayed as a percentage (e.g., "327% of Medicare rate")
- ☐ The Medicare locality used for comparison is identified (e.g., "CMS Locality 07 — Rest of South Carolina")
- ☐ Source citation links to the current CMS Physician Fee Schedule

US-008 — Detect duplicate charges automatically

As a repeat patient user who disputes billing errors, I want to have the system automatically detect when the same CPT code is billed twice on the same date, so that I don't have to manually scan line items to catch a duplication the hospital may claim was accidental.

Priority: Must Have | **Points:** 3 | **Sprint:** 3 | **Epic:** Knowledge Base & Error Detection

Requirements: FR-11, FR-16

Acceptance Criteria:

- ☐ System flags any CPT code that appears more than once with the same date of service
- ☐ Both affected line items are identified by line number in the error card
- ☐ Estimated savings equals the cost of the duplicate line item
- ☐ Module correctly ignores the same CPT code billed on different dates (not a duplicate)
- ☐ Unit tests pass for: exact duplicate (flag), same code different date (no flag), same code different room (flag with note)

US-009 — Get regulatory citations I can use in a formal dispute

As a repeat patient user who disputes billing errors, I want to receive specific regulatory references (No Surprises Act, CMS guidelines, MPFS) in the error report and dispute letter, so that my dispute carries legal weight and the hospital billing department takes it seriously.

Priority: Must Have | **Points:** 3 | **Sprint:** 4 | **Epic:** RAG Integration & Dispute Letters

Requirements: FR-17, FR-18, FR-22

Acceptance Criteria:

- ☐ Every error explanation includes at least one citation with: source name, document title, section reference
- ☐ Citations link to the publicly accessible source document (CMS.gov, Congress.gov)

- ☐ Dispute letter includes regulatory citations in the body text, not just as footnotes
- ☐ RAG explanations are grounded — every factual claim supported by a retrieved knowledge base passage
- ☐ Citation accuracy is measurable and documented in the evaluation report (target >85%)

US-010 — Reconcile my bill against my insurance EOB

As a repeat patient user who disputes billing errors, I want to upload both my provider bill and my insurance Explanation of Benefits, so that the system can cross-reference them and flag any discrepancies between what the provider billed and what the insurer processed.

Priority: Should Have | **Points:** 8 | **Sprint:** 3 | **Epic:** Knowledge Base & Error Detection

Requirements: FR-03, FR-13

Acceptance Criteria:

- ☐ Upload page offers an optional second upload zone labelled "EOB (optional — for enhanced analysis)"
- ☐ When EOB is uploaded, Module 3 runs and cross-references line items by date, CPT code, and amount
- ☐ Mismatches are flagged with both the bill value and the EOB value shown side by side
- ☐ If no EOB is uploaded, Module 3 is skipped gracefully — no error shown to user
- ☐ Fuzzy matching handles minor formatting differences between bill and EOB (e.g., date format variations)

US-011 — Detect potential No Surprises Act violations

As a repeat patient user who disputes billing errors, I want to be alerted when my bill may contain a balance billing charge prohibited under the No Surprises Act, so that I can dispute charges from out-of-network providers in contexts where federal law protects me.

Priority: Should Have | **Points:** 5 | **Sprint:** 3 | **Epic:** Knowledge Base & Error Detection

Requirements: FR-14, FR-17

Acceptance Criteria:

- ☐ Module 4 identifies out-of-network provider flags in EOB data
- ☐ Module cross-references service context (emergency care, certain outpatient settings) against NSA applicability
- ☐ Flagged violations include a plain-English explanation of what the No Surprises Act says and why it applies
- ☐ Citation links directly to the No Surprises Act text on Congress.gov and relevant CMS guidance
- ☐ Module correctly does not flag out-of-network charges where NSA does not apply (e.g., elective outpatient)

US-012 — Download results as a PDF for my records

As a repeat patient user who disputes billing errors, I want to download the full error analysis report as a PDF, so that I have a permanent record of what was found, the evidence, and the citations — separate from the dispute letter.

Priority: Should Have | **Points:** 3 | **Sprint:** 4 | **Epic:** RAG Integration & Dispute Letters

Requirements: FR-21, FR-23

Acceptance Criteria:

- ☐ "Download Results PDF" button appears on the error report page
- ☐ PDF includes: summary of errors, each error card with explanation and citation, total savings estimate, date of analysis
- ☐ PDF is formatted professionally (headers, sections, MediCheck branding)
- ☐ PDF is generated within 5 seconds of clicking the button
- ☐ Downloaded filename includes the date: MediCheck_Analysis_YYYY-MM-DD.pdf

Persona 3 — Jordan: Clinic Integration Developer

Jordan is a full-stack developer on the Atrium Health patient portal team in Charlotte, NC. Needs to integrate MediCheck's analysis capabilities into an existing patient portal within a single sprint.

US-013 — Call the bill analysis endpoints from an external React app

As a clinic integration developer, I want to call POST /upload, POST /confirm, and POST /analyse from our existing React + Node.js portal, so that I can surface MediCheck's error detection inside our patient portal without rebuilding the analysis logic.

Priority: Should Have | **Points:** 5 | **Sprint:** 2 | **Epic:** Three-Service Infrastructure

Requirements: FR-04, FR-05, FR-08, FR-26, NFR-15, NFR-16

Acceptance Criteria:

- ☐ POST /upload, POST /confirm, and POST /analyse accept requests from an external origin (CORS configured)
- ☐ All endpoints return consistent JSON response structures with documented field names and types
- ☐ All endpoints return appropriate HTTP status codes (200, 400, 422, 500, 503)
- ☐ A README section documents each endpoint with example request bodies and response payloads
- ☐ Postman collection committed to /docs/postman/ for integration testing

US-014 — Receive structured JSON with error details I can render in our UI

As a clinic integration developer, I want to receive a structured JSON response from POST /analyse that includes error type, severity, affected line items, and estimated dollar impact, so that I can map the response directly to UI components without parsing unstructured text.

Priority: Should Have | **Points:** 3 | **Sprint:** 3 | **Epic:** Knowledge Base & Error Detection

Requirements: FR-09, FR-16, NFR-15, NFR-16

Acceptance Criteria:

- ☐ POST /analyse response includes a top-level errors array
- ☐ Each error object contains: module, error_type, description, line_items_affected (array), estimated_dollar_impact (float), confidence (float 0–1), explanation (string), citations (array)
- ☐ Response includes top-level total_errors (int) and total_savings (float)
- ☐ Schema is consistent — missing optional fields use empty arrays not null
- ☐ Schema is documented in the README with a full example response

US-015 — Handle Service 3 timeouts gracefully in the portal

As a clinic integration developer, I want to receive a meaningful partial response when the RAG service (Service 3) times out rather than a 500 error, so that the patient portal can show error detection results even if plain-English explanations are temporarily unavailable.

Priority: Must Have | **Points:** 3 | **Sprint:** 3 | **Epic:** Three-Service Infrastructure

Requirements: FR-19, FR-20, FR-26, NFR-02, NFR-11, NFR-16, NFR-17, NFR-18, NFR-26

Acceptance Criteria:

- ☐ If Service 3 does not respond within 10 seconds, Service 2 returns detection results with explanation: null and citations: []
- ☐ The response body includes a top-level rag_available: false flag
- ☐ Service 2 logs the timeout internally but does not expose internal error details in the response body

- ☐ A retry endpoint (POST /explain/{session_id}) allows the portal to fetch explanations when Service 3 recovers
- ☐ Integration test covers the timeout scenario using a mocked Service 3 that returns no response

US-016 — Monitor service health from our portal's infrastructure

As a clinic integration developer, I want to call a /health endpoint on both Service 2 and Service 3, so that our portal's monitoring system can detect MediCheck outages and display a maintenance message instead of broken UI.

Priority: Must Have | **Points:** 2 | **Sprint:** 2 | **Epic:** Three-Service Infrastructure

Requirements: FR-20, NFR-04, NFR-11, NFR-12, NFR-13, NFR-24

Acceptance Criteria:

- ☐ GET /health on Service 2 returns {"status": "ok", "service": "bill-analysis", "version": "1.0.0"} with HTTP 200
- ☐ GET /health on Service 3 returns {"status": "ok", "service": "rag", "version": "1.0.0"} with HTTP 200
- ☐ Both endpoints respond within 1 second under normal conditions
- ☐ GitHub Actions post-deploy step pings both /health endpoints and fails if either returns non-200
- ☐ Health endpoint does not require authentication

US-017 — Integrate using clear documentation and example payloads

As a clinic integration developer, I want to find complete API documentation with example request and response payloads in the repository, so that I can complete integration in a single sprint without needing to contact the MediCheck team for clarification.

Priority: Should Have | **Points:** 3 | **Sprint:** 5 | **Epic:** Final Presentation & Submission

Requirements: NFR-07, NFR-09, NFR-14

Acceptance Criteria:

- ☐ README.md includes an API reference section: method, path, request body schema, response body schema, error codes
- ☐ At least one complete worked example per endpoint (realistic request + realistic response)
- ☐ Error codes documented: what each HTTP status means in context (e.g., 422 = field validation failed)
- ☐ .env.example documents all required environment variables with comments
- ☐ Setup instructions allow a developer to run all three services locally in under 15 minutes

US-018 — Validate that the CI/CD pipeline is stable before integrating

As a clinic integration developer, I want to see that the MediCheck repository has automated tests running on every change and auto-deploys only when tests pass, so that I can trust that the deployed service URLs are stable and won't break our portal with an untested change.

Priority: Should Have | **Points:** 3 | **Sprint:** 2 | **Epic:** Three-Service Infrastructure

Requirements: FR-15, NFR-09, NFR-11, NFR-18, NFR-23, NFR-24, NFR-26

Acceptance Criteria:

- ☐ GitHub Actions CI badge is visible in the README showing current build status
- ☐ Every PR to main triggers: dependency install, pytest run, black formatting check, pylint check
- ☐ Merge to main only triggers a Render redeploy if all tests pass

- ☐ Post-deploy step pings /health on all three services and fails if any return non-200
- ☐ Build history is publicly visible in GitHub Actions so integration developers can see recent test results

3. User Journey

Primary Persona: Sarah (First-Time Patient User) | Journey: End-to-End Bill Analysis Flow

Goal: Determine if her $2,400 MUSC Health bill contains errors and get evidence to dispute any overcharges.

Journey Duration: 5–10 minutes | Success Metric: Sarah identifies $747 in billing errors and downloads a dispute letter.

Phase	Steps	Key Actions	Services Involved
Phase 1: Discovery & Onboarding	Steps 1–3	Google search → Landing page → Upload page	Service 1 (React frontend)
Phase 2: Upload & Extraction	Steps 4–6	File upload → OCR processing → Field confirmation	Service 1 → Service 2 (Flask + AWS Textract)
Phase 3: Analysis & Results	Steps 7–8	Error detection → Error report display	Service 2 → Service 3 (RAG / GPT-4o-mini)
Phase 4: Action & Resolution	Steps 9–12	Review details → Generate letter → Download → Follow-up	Service 2 → Service 3 (python-docx / ReportLab)

Phase 1 — Discovery & Onboarding

Step 1 — Google Search

Sarah searches "check medical bill for errors free." Emotional state: Frustrated — received a confusing bill, doesn't know where to start.

Pain points: Overwhelmed by billing complexity, skeptical of online tools (privacy concerns), worried about cost of professional auditing.

Step 2 — Landing Page

Reads headline "AI-Powered Medical Bill Error Detection — Free Analysis." Sees example: "We found $847 in errors in this $2,400 MUSC Health bill." Emotional state: Curious — sounds too good to be true, but hopeful.

Decision Point 1: Should I trust this tool with my medical bill?

Trust Factor	Effect
Privacy statement: "We don't store your data"	✅ Trust builder
Example results citing specific SC providers	✅ Trust builder
Free, no account required	✅ Trust builder
Unknown brand	❌ Trust reducer

Step 3 — Upload Page

Clicks "Analyse Your Bill" CTA. Arrives at upload page. Emotional state: Tentative — ready to try, still cautious.

Reassurance shown: "Your bill is processed securely and deleted after 24 hours" | Supported formats: PDF, JPG, PNG (up to 10 MB)

Phase 2 — Upload & Extraction

Step 4 — File Upload

Drags musc_health_bill.pdf to upload zone. POST /upload sent to Service 2. Service 2 saves file, returns session ID. Emotional state: Anxious — committed, waiting to see if it works.

Error Scenario	System Response
File > 10 MB	File too large. Please try a smaller file or photo.
Corrupt file	Unable to read file. Please try a different version.
Network timeout	Upload timed out. Check your connection and retry.
Wrong format (.docx)	Immediate client-side: Unsupported file type. Please upload PDF, JPG, or PNG.

Step 5 — OCR Processing

Service 2 calls AWS Textract AnalyzeDocument API (~10 seconds). Extracts text, tables, key-value pairs. Runs spaCy NER for dates, amounts, CPT codes. Emotional state: Waiting — trust builds if progress is visible.

Step 6 — Field Confirmation

Reviews extracted data in editable table. Fields with confidence <80% highlighted in yellow. Corrects one low-confidence amount. Clicks Confirm & Analyse. Emotional state: Engaged — actively participating, feels in control.

Field	Extracted Value	Confidence	Action
Date of Service	03/15/2024	92%	✓ Confirmed
Provider	MUSC Health University Medical Center	98%	✓ Confirmed
Procedure	Knee Arthroscopy	89%	✓ Confirmed
Amount	$2,40 ⚠️	73%	Sarah corrects to $2,400

Phase 3 — Analysis & Results

Step 7 — Running Analysis

Service 2 calls ErrorDetectionEngine.run(confirmed_fields). Module 1: Finds CPT 29881 billed twice → $480 duplicate. Module 2: CPT 99215 charged $267 vs. Medicare rate $82 (327% outlier). Module 3: No EOB → skipped. Module 4: No out-of-network flags → skipped. Service 2 calls Service 3 POST /explain for each error. GPT-4o-mini generates explanations at temperature=0.

Error Scenario	System Response
Service 3 timeout (>10 seconds)	Errors returned without explanations; rag_available: false; Retry button shown
No errors detected	Great news! We didn't detect any obvious billing errors in your bill.
LLM rate limit	Generic fallback text shown; raw detection output preserved

Step 8 — Error Report Display

Sarah sees savings summary: "2 errors found · $747 potential savings." Emotional state: Surprised → Relieved — errors confirmed. I was right to be suspicious.

Error Card 1 — Red (High Severity): Duplicate Charge · $480 savings. Line items 3 and 8 both bill CPT 29881 on 03/15/2024. Source: CMS Medicare Claims Processing Manual, Chapter 23.

Error Card 2 — Amber (Medium Severity): Medicare Rate Outlier · $267 savings. CPT 99215 charged $267 vs. Medicare rate $82 (327%). Source: CMS Physician Fee Schedule, 2024, Locality 07.

Phase 4 — Action & Resolution

Step 9 — Review Errors in Detail

Clicks "Show more details" on duplicate charge. Sees side-by-side comparison of line items 3 and 8. Clicks "Learn more" → opens CMS guideline in new tab. Emotional state: Empowered — learning, feels able to challenge the hospital.

Step 10 — Generate Dispute Letter

Clicks "Generate Dispute Letter." Service 2 calls Service 3 POST /draft-letter. LLM generates letter content grounded in knowledge base at temperature=0. Service 2 formats via python-docx template and generates PDF via ReportLab. Emotional state: Confident — professional letter, ready to send.

Decision Point 4: Sarah wants to add her phone number. Downloads Word version, adds it, then sends to MUSC Health billing.

Step 11 — Download & Send

Downloads MediCheck_Dispute_Letter_2024-03-16.docx. Opens in Microsoft Word. Adds phone number. Attaches to email to MUSC Health billing. Sends. Emotional state: Satisfied → Empowered.

Metric	Result
Time from upload to letter sent	8 minutes
Errors identified	2
Potential savings	$747
User confidence level	High — evidence-based dispute

Step 12 — Follow-Up (1 Week Later)

Receives response from MUSC Health. Duplicate charge ($480) removed. New bill: $1,920. Sarah now checks every bill proactively.

Emotional Journey

Step	Description	Emotional State
S1	Google Search	😟 Frustrated
S2	Landing Page	🤔 Curious
S3	Upload Page	😐 Tentative
S4	File Upload	😬 Anxious
S5	OCR Processing	⏳ Waiting
S6	Field Confirmation	🧐 Engaged
S7	Analysis Running	😯 Anticipating
S8	Error Report	😲 Surprised → 😊 Relieved
S9	Error Details	🤓 Empowered
S10	Letter Generated	😌 Confident
S11	Download & Send	😊 Satisfied
S12	Follow-Up Success	💪 Empowered

4. System Requirements

Version 4.0 | Sprint 1 | Owner: Member 2 (Scrum Master + Code Owner)

4.1 Functional Requirements

FR — Document Upload

ID	Requirement
FR-01	The system shall accept PDF document uploads with a maximum file size of 10 MB and a maximum document length of 20 pages. Files exceeding either limit shall be rejected before processing begins.
FR-02	The system shall extract structured text and key-value data from uploaded PDFs automatically — including at minimum: patient name, provider name, date of service, and all line items with procedure codes and billed amounts. No manual data entry shall be required.
FR-03	The system shall support upload of two documents per session: one provider bill and one EOB. If the user attempts to trigger analysis with zero documents, the system shall return an error and refuse to proceed.
FR-04	The system shall generate and return a unique UUID session identifier upon successful upload, included in every subsequent request referencing that session.
FR-05	The system shall validate every upload request and return HTTP 400 with a structured error body (error code + human-readable description) if the file is not a PDF, exceeds 10 MB, or exceeds 20 pages.

FR — Field Confirmation

ID	Requirement
FR-06	The system shall display all extracted fields before analysis is permitted. The user shall not be able to proceed to analysis without visiting this page.
FR-07	The system shall allow the user to edit any extracted field. Every edited value shall replace the original for all downstream processing; the original shall be retained for audit purposes.
FR-08	Upon submission of confirmed fields, the system shall persist the confirmed data linked to the session ID and return HTTP 200. Analysis requests submitted before confirmation shall return HTTP 400.
FR-09	The system shall return HTTP 404 if a field confirmation submission references a session ID that does not exist.

FR — Error Detection

ID	Requirement
FR-10	The system shall execute all four error detection checks on every analysis request. Each check shall produce either an error finding or an explicit all-clear — no check may be silently omitted.

ID	Requirement
FR-11	The duplicate charge detection check shall identify every instance where the same procedure code appears more than once for the same date of service. Each duplicate pair shall produce a separate error result.
FR-12	The rate outlier check shall compare each billed amount against the CMS Physician Fee Schedule locality-adjusted Medicare rate. Any line item exceeding 300% of the applicable Medicare rate shall produce an error result including billed amount, Medicare rate, and percentage.
FR-13	The EOB reconciliation check shall compare every line item in the confirmed provider bill against the confirmed EOB. Any discrepancy in procedure code, date, quantity, or billed amount shall produce a separate error result.
FR-14	The NSA violation check shall produce an error result for every line item where the provider is out-of-network and the service context meets federal criteria for balance billing prohibition. The result shall identify the provider, network status, and applicable federal protection.
FR-15	The error detection pipeline shall be designed such that a new check can be added without modifying any existing check or pipeline orchestration logic (Strategy pattern). This shall be demonstrated by the ability to add a fifth check without touching existing code.
FR-16	Every error result shall include: detection check name, error category, plain-English description, affected line items (line number + procedure code), estimated dollar impact (USD), and confidence level (high/medium/low). Any missing field = system defect.

FR — Knowledge-Grounded Explanations

ID	Requirement
FR-17	The system shall maintain a knowledge base from four CMS sources: CMS Physician Fee Schedule, ICD-10-CM guidelines, No Surprises Act regulatory text + CMS guidance, and the CMS Procedure-to-RVU crosswalk. All four sources shall be present and queryable before the explanation service is ready.
FR-18	For each error result, the system shall produce a plain-English explanation retrieved from the knowledge base, including at least one citation (source document name + section/page). An explanation with no citation = system defect.
FR-19	The explanation service shall return an error response — not a generated explanation — for any query outside the medical billing domain, with no exceptions based on phrasing.
FR-20	The explanation service shall be hosted independently of the error detection service. A redeployment or restart of the explanation service shall have no effect on the error detection pipeline, and vice versa.

FR — Dispute Letter Generation

ID	Requirement
FR-21	The system shall generate a formal dispute letter on request for any session with completed analysis. The letter shall be produced in both Word (.docx) and PDF formats, both available for download within the same response.
FR-22	The generated letter shall contain: patient name + insurance member ID, provider name + identifier, session reference ID, generation date, numbered list of every detected error (procedure code, billed amount, estimated overcharge, regulatory citation), total estimated overcharge, and a formal dispute request paragraph. Missing any element = system defect.

ID	Requirement
FR-23	Both Word and PDF versions shall be retrievable within the session without re-uploading, re-confirming fields, or repeating the analysis.

FR — Session Management

ID	Requirement
FR-24	The system shall not require users to register, provide an email, or supply credentials. A session is initiated solely by a successful document upload with no prior authentication step.
FR-25	The system shall persist all session data (uploaded documents, confirmed fields, analysis results, dispute letters) in a durable relational data store for a minimum of 24 hours after session creation.
FR-26	Each session shall have a status field transitioning in order: document uploaded → fields confirmed → analysis complete → letter generated. Out-of-sequence requests shall return HTTP 400.
FR-27	At session creation, the system shall display the session ID and a clear message stating it is the sole means of referencing results, visible until the user explicitly proceeds.

4.2 Non-Functional Requirements

NFR — Performance & Latency

ID	Requirement
NFR-01	The system shall return a complete analysis response — all four detection check results and all knowledge-grounded explanations — within 30 seconds for a document up to 20 pages, under normal operating conditions with no more than 10 concurrent users.
NFR-02	If the explanation service does not respond within 10 seconds, the bill analysis service shall return a partial response containing detection results and a flag (rag_available: false) indicating explanations are unavailable. The caller shall receive this within 2 seconds of the 10-second timeout.
NFR-03	The system shall return a session identifier within 60 seconds of receiving a document upload request for up to 20 pages, under normal operating conditions.
NFR-04	Each health check endpoint shall return HTTP 200 within 2 seconds under normal conditions. A response exceeding 2 seconds shall be treated as a health check failure by the CI/CD pipeline.
NFR-05	The explanation service shall achieve p50 latency <10 seconds and p95 latency <20 seconds, validated against a minimum of 15 test queries during Sprint 5.

NFR — Security & Data Handling

ID	Requirement
NFR-06	No real patient data shall be used at any stage of development, testing, or demonstration. All test and demo documents shall use synthetic data with a visible disclaimer on the first page.
NFR-07	All credentials and secret configuration values (API keys, database connection strings, inter-service URLs) shall be stored as environment variables. Zero secrets shall be committed to the source code repository. A reference config file listing all required variable names (without values) shall be in the repo root.
NFR-08	All communication between the browser and the application, and all inter-service communication, shall use HTTPS. Requests over plain HTTP shall be redirected or rejected.
NFR-09	All service-to-service addresses shall be read from environment config at runtime. Any pull request containing a hardcoded service address shall be blocked from merging.
NFR-10	No copyrighted procedure code descriptions shall appear in any stored data, API response, UI element, or generated document. All logic shall reference procedure codes by numeric identifier only, using publicly available CMS rate data.

NFR — Deployment & Availability

ID	Requirement
NFR-11	Each functional capability shall be independently deployable, with its own publicly accessible URL and independently managed environment variables. Redeploying one capability shall not cause downtime in any other.
NFR-12	Each backend service shall expose GET /health returning HTTP 200 with a JSON body containing at minimum status: "ok" and a service name field. This endpoint shall be implemented and responding before any other endpoint is considered ready.

ID	Requirement
NFR-13	The knowledge base used by the explanation service shall be stored on a persistent volume that survives service restarts. Following a restart, the explanation service shall serve explanation requests without re-ingesting any source document.
NFR-14	The total infrastructure cost of all deployed services shall not exceed $0/month for a prototype deployment on free-tier limits. A cost analysis comparing free-tier vs. estimated production-scale costs shall be documented in the design and testing document.

NFR — API Contract

ID	Requirement
NFR-15	All inter-service request and response payloads shall use JSON with Content-Type: application/json. The schema for every inter-service payload shall be defined and documented in a single shared location. Any deviation = defect.
NFR-16	All API endpoints shall return error responses in a consistent structure containing at minimum: a machine-readable error code (string), a human-readable message (string), and the session ID (string or null). Every endpoint shall conform without exception.
NFR-17	The system shall return HTTP 400 for any analysis request submitted before field confirmation. The system shall return HTTP 404 for any letter generation request for a session with no completed analysis. Both conditions shall be integration-tested.
NFR-18	Every outbound HTTP call from the bill analysis service to the explanation service shall specify an explicit 10-second timeout. Absence of an explicit timeout on any such call = code review failure, blocking the PR.

NFR — UI & Accessibility

ID	Requirement
NFR-19	The user interface shall apply a consistent visual design — typography, colour palette, spacing, and component styling — across all four pages: document upload, field confirmation, error report, and dispute letter download. Visual inconsistencies = defects.
NFR-20	Error report results shall use both a colour indicator and a text label for severity. High-impact (>$500): red, "High Impact". Medium-impact ($100–$500): amber, "Medium Impact". Informational (<$100): green, "Informational". All three levels shall be present and correctly applied in test scenario output.
NFR-21	Every input field on the field confirmation page shall have a visible label. Required fields left blank when the user attempts to proceed shall display an inline error message identifying the field and reason. The system shall prevent navigation to analysis until all required fields contain a non-empty value.
NFR-22	The user interface shall be responsive and usable across desktop, tablet, and mobile without horizontal scrolling or overlapping content. Responsive behaviour shall be verified against at minimum three screen size categories as part of the Sprint 5 review.

NFR — Testing & CI/CD

ID	Requirement
NFR-23	The automated build and test pipeline shall execute on every pull request and every push to main. The pipeline shall: install all dependencies, run the full test suite, and run a lint check. Any pull request where one or more steps fail shall be blocked from merging without exception.
NFR-24	On a successful pipeline run triggered by a push to main, all deployed services shall be automatically redeployed in sequence. Following redeployment, the pipeline shall call each backend service's health check endpoint and verify HTTP 200 within 5 seconds. Failure to return HTTP 200 within 5 seconds = pipeline failure, logged in the CI/CD run log.
NFR-25	The test suite shall include a minimum of 15 unit tests distributed across all four error detection checks, with at least 3 tests per check: one positive case, one negative case, and one boundary case. The suite shall also include integration tests for the end-to-end pipeline and smoke tests for all service health check endpoints.
NFR-26	Every integration test exercising a code path involving a call from the bill analysis service to the explanation service shall use a simulated response. The test suite shall pass without a running instance of the explanation service. Any integration test making a live HTTP call = code review rejection.
NFR-27	A test coverage report shall be generated as part of the Sprint 5 test run and the summary committed to the design and testing document. Line coverage across all four error detection checks shall be a minimum of 80%. Coverage below 80% on any check = Sprint 5 acceptance failure requiring remediation before code freeze.

5. Requirements Traceability Matrix

Full bidirectional coverage: all 54 requirements (27 FR + 27 NFR) linked to at least one user story. All 18 user stories linked to at least one requirement. Zero orphans in either direction.

Category	Count	Linked to Stories	Orphaned
Functional Requirements (FR)	27	27	0
Non-Functional Requirements (NFR)	27	27	0
Total Requirements	54	54	0
User Stories	18	18	0

5.1 Functional Requirements → User Stories

Req ID	Description	Linked Stories	Status
FR-01	PDF upload — max 10 MB and 20 pages	US-001	Planned
FR-02	Automatic extraction of 4 field categories; no manual entry	US-001, US-002	Planned
FR-03	Two documents per session; analysis blocked if zero documents	US-001, US-010	Planned
FR-04	UUID session identifier returned in every subsequent response	US-006, US-013	Planned
FR-05	HTTP 400 with error code + description on upload validation failure	US-001, US-013	Planned
FR-06	All extracted fields displayed before analysis; step cannot be bypassed	US-002	Planned
FR-07	Every field editable; edited values replace originals; originals retained	US-002	Planned
FR-08	Confirmed fields persisted; HTTP 400 if analysis before confirmation	US-002, US-013	Planned
FR-09	HTTP 404 if confirmation references non-existent session	US-013, US-014	Planned
FR-10	All four checks executed every time; each produces result or all-clear	US-003, US-007, US-008, US-010, US-011	Planned
FR-11	Duplicate charge detection — each duplicate pair = separate result	US-008	Planned
FR-12	Rate outlier — >300% Medicare rate flagged with amounts and percentage	US-007	Planned
FR-13	EOB reconciliation — each field discrepancy = separate result	US-010	Planned

Req ID	Description	Linked Stories	Status
FR-14	NSA violation — identifies provider, network status, applicable protection	US-011	Planned
FR-15	Fifth check addable without modifying existing checks (Strategy pattern)	US-018	Planned
FR-16	Every error result includes all six required fields; missing = defect	US-004, US-007, US-008, US-009, US-014	Planned
FR-17	All four CMS sources present and queryable before service is ready	US-007, US-009, US-011	Planned
FR-18	Each explanation includes at least one citation; zero citations = defect	US-004, US-009	Planned
FR-19	Out-of-domain queries return error response; no exceptions	US-015	Planned
FR-20	Explanation service restart has no effect on detection pipeline	US-015, US-016	Planned
FR-21	Both Word (.docx) and PDF produced in single request	US-005, US-012	Planned
FR-22	Letter contains all required elements; any missing element = defect	US-005, US-009	Planned
FR-23	Both formats retrievable without repeating prior steps	US-005, US-012	Planned
FR-24	No registration or credentials required; session initiated by upload only	US-006	Planned
FR-25	All session data retrievable by session ID for minimum 24 hours	US-006	Planned
FR-26	Session status transitions in order; out-of-sequence requests = HTTP 400	US-013, US-015	Planned
FR-27	Session ID and recovery warning displayed before user proceeds from upload	US-006	Planned

5.2 Non-Functional Requirements → User Stories

Req ID	Description	Linked Stories	Status
NFR-01	Full analysis within 30 seconds; up to 20 pages; up to 10 concurrent users	US-003	Planned
NFR-02	Partial response within 2 seconds of 10-second explanation timeout	US-003, US-015	Planned
NFR-03	Session identifier returned within 60 seconds of upload request	US-001, US-006	Planned
NFR-04	Health check response within 2 seconds; exceeding limit = pipeline failure	US-016	Planned
NFR-05	Explanation latency p50 <10s, p95 <20s; validated over minimum 15 queries	US-003, US-009	Planned
NFR-06	Synthetic data only; every test document includes visible disclaimer	US-006	Planned
NFR-07	Zero secrets in repository; reference config file in repo root	US-006, US-017	Planned
NFR-08	All communication over HTTPS; plain HTTP rejected or redirected	US-006	Planned
NFR-09	All service addresses from environment config; hardcoded addresses block PR	US-017, US-018	Planned
NFR-10	No copyrighted CPT descriptions in any stored data, response, or document	US-007, US-008	Planned
NFR-11	Each capability independently deployable; own URL and environment config	US-015, US-016, US-018	Planned
NFR-12	GET /health returns HTTP 200 with status and service name before other endpoints	US-016	Planned
NFR-13	Knowledge base survives restarts; ready without re-ingestion	US-015, US-016	Planned
NFR-14	Prototype cost $0/month; free-tier vs production cost analysis documented	US-017	Planned
NFR-15	All inter-service payloads JSON; schema defined in one place; deviation = defect	US-013, US-014	Planned
NFR-16	All error responses include error code, message, and session ID	US-013, US-014, US-015	Planned
NFR-17	HTTP 400 before confirmation; HTTP 404 for missing results; both integration-tested	US-013, US-015	Planned
NFR-18	All explanation service calls have explicit 10-second timeout; missing = PR block	US-015, US-018	Planned
NFR-19	Consistent design across all four pages; inconsistencies = defects	US-002, US-004	Planned
NFR-20	Three severity levels with correct colour and text label; verified in test output	US-004, US-007	Planned
NFR-21	All fields labelled; required fields block progression until non-empty	US-002	Planned

Req ID	Description	Linked Stories	Status
NFR-22	Responsive across mobile, tablet, desktop; verified in Sprint 5 review	US-001, US-002, US-004, US-005	Planned
NFR-23	Pipeline runs on every PR and push to main; failing step blocks merge	US-018	Planned
NFR-24	Auto-deploy on passing main; health checks within 5 seconds post-deploy	US-016, US-018	Planned
NFR-25	Minimum 15 unit tests; 3 per check (positive, negative, boundary)	US-008, US-010, US-011, US-012	Planned
NFR-26	All inter-service integration tests use simulated responses; live calls rejected	US-015, US-018	Planned
NFR-27	Coverage report committed; minimum 80% per detection check	US-007, US-008, US-010, US-011	Planned

5.3 Reverse Matrix — User Story → Requirements

Story ID	Persona	Title	Linked Requirements	Req Count
US-001	Sarah	Upload a medical bill	FR-01, FR-02, FR-03, FR-05, NFR-03, NFR-22	6
US-002	Sarah	Review extracted fields	FR-02, FR-06, FR-07, FR-08, NFR-19, NFR-21, NFR-22	7
US-003	Sarah	Results within 30 seconds	FR-10, NFR-01, NFR-02, NFR-05	4
US-004	Sarah	Errors in plain English	FR-16, FR-18, NFR-19, NFR-20, NFR-22	5
US-005	Sarah	Download dispute letter	FR-21, FR-22, FR-23, NFR-22	4
US-006	Sarah	Data privacy	FR-04, FR-24, FR-25, FR-27, NFR-03, NFR-06, NFR-07, NFR-08	8
US-007	Marcus	Medicare rate comparison	FR-10, FR-12, FR-16, FR-17, NFR-10, NFR-20, NFR-27	7
US-008	Marcus	Duplicate charge detection	FR-10, FR-11, FR-16, NFR-10, NFR-25, NFR-27	6
US-009	Marcus	Regulatory citations	FR-16, FR-17, FR-18, FR-22, NFR-05	5
US-010	Marcus	EOB reconciliation	FR-03, FR-10, FR-13, NFR-25, NFR-27	5
US-011	Marcus	NSA violations	FR-10, FR-14, FR-17, NFR-25, NFR-27	5
US-012	Marcus	Download results PDF	FR-21, FR-23, NFR-25	3
US-013	Jordan	External app endpoints	FR-04, FR-05, FR-08, FR-09, FR-26, NFR-15, NFR-16, NFR-17	8
US-014	Jordan	Structured JSON response	FR-09, FR-16, NFR-15, NFR-16	4
US-015	Jordan	Service 3 timeout handling	FR-19, FR-20, FR-26, NFR-02, NFR-11, NFR-13, NFR-16, NFR-17, NFR-18, NFR-26	10
US-016	Jordan	Health endpoint monitoring	FR-20, NFR-04, NFR-11, NFR-12, NFR-13, NFR-24	6
US-017	Jordan	API documentation	NFR-07, NFR-09, NFR-14	3
US-018	Jordan	CI/CD pipeline stability	FR-15, NFR-09, NFR-11, NFR-18, NFR-23, NFR-24, NFR-26	7

5.4 Gap Analysis

Requirements with single-story coverage — higher risk if that story is descoped.

Req ID	Description	Covered By
FR-06	All extracted fields displayed before analysis	US-002 only
FR-07	Every field editable; originals retained	US-002 only
FR-11	Duplicate charge detection	US-008 only
FR-12	Rate outlier detection (>300% Medicare)	US-007 only
FR-13	EOB reconciliation	US-010 only
FR-14	NSA violation detection	US-011 only
FR-15	New check addable without modifying existing (Strategy pattern)	US-018 only
FR-19	Out-of-domain queries return error	US-015 only
FR-24	No registration required	US-006 only
FR-27	Session ID warning displayed at upload	US-006 only
NFR-01	Full analysis within 30 seconds	US-003 only
NFR-04	Health check within 2 seconds	US-016 only
NFR-12	GET /health implemented before other endpoints	US-016 only
NFR-14	Prototype cost $0/month; cost analysis documented	US-017 only
NFR-21	All fields labelled; required fields block progression	US-002 only
NFR-23	Pipeline runs on every PR and push to main	US-018 only

Stories covering the most requirements (highest implementation complexity):

User Story	Req Count	Risk Level
US-015 — Handle Service 3 timeouts (Jordan)	10	🔴 High
US-013 — External app endpoints (Jordan)	8	🔴 High
US-006 — Data privacy (Sarah)	8	🔴 High
US-002 — Review extracted fields (Sarah)	7	🟡 Medium-High
US-007 — Medicare rate comparison (Marcus)	7	🟡 Medium-High
US-018 — CI/CD pipeline stability (Jordan)	7	🟡 Medium-High